

Code review — recapitulare-vectori

Runde de review pe proiectul tău, cea mai nouă sus.

Runda 1 — 2026-10-01 — sha 1e854cc

Revizuit: `recapitulare-vectori.h` (648 de linii, 34 de funcții, exercițiile `sol1` – `sol19`) și `recapitulare-vectori.cpp`.

Poarta de compilare:  **trece.** `g++ -std=c++20 -Wall -Wextra` compilează fără erori, cu **2 warning-uri** — amândouă sunt constatări reale mai jos (B7 și B5):

```
recapitulare-vectori.h:400: warning: non-void function does not return
                             a value in all control paths
recapitulare-vectori.h:647: warning: unused parameter 'm'
```

Verificarea s-a făcut prin rulare, nu prin citire. Am copiat headerul într-un proiect separat și am chemat funcțiile pe valori alese special pentru capete (`0`, numere cu zerouri la mijloc, vectori dimensionați exact). Unde am bănuir o depășire de vector, am rulat sub AddressSanitizer, ca să nu rămână la nivel de presupunere.

Întâi ce ai închis — și e mult

Ai reparat singură amândouă problemele din afisare. Pe 24 septembrie la 19:47 aveai `for (int i = 1; i <= n; i++)` și `cout << v[i] << ""`. La 21:01 — la **o oră** după ce ai primit PDF-ul rundei 1 de pe `recapitulare-cpp` — ai comis varianta corectă:

```
for (int i = 0; i < n; i++) {
    cout << v[i] << " ";
}
```

Nimeni nu ți-a spus să repari `afisare` ; constatarea aia era despre alt repo și alte funcții. Ai citit explicația cu `i = 0; i < n` , ai recunoscut aceeași formă aici și ai aplicat-o. Țasta e exact pasul al doilea care-ți lipsea — merită spus, pentru că e lucrul greu.

Și ai închis M1 din runda trecută: funcțiile tale se cheamă între ele. `ctPrimeVec` cheamă `nrPrim` , `cifMinimaVect` cheamă `cifMinima` , `afisareNumereCeContinCifraMinima` cheamă și `cifMinimaVect` și `contineCifra` , `cifreMijlocPatrate` cheamă `ePatratPerfect` , `cmmmc` cheamă `cmmdc` . Pe fișa 2 nu exista niciun apel între funcțiile tale. Acum e peste tot, și se vede în cât de scurte au devenit funcțiile de vector.

Corecte cap-coadă, verificate prin rulare: `nrPrim` , `rasturnat` (inclusiv `rasturnat(100) → 1`), `cifraControl` (inclusiv pe `0` și pe `9999`), `prodCifre` , `eSuperprim` , `cifMax` , `palindrom` , `cmmdc2` .

`cmmdc2` merită o vorbă în plus: ai scris algoritmul lui Euclid **cu rest**, lângă cel cu scăderi repetate. Țla cu rest e varianta bună, și nu ți-a cerut-o nimeni.

● Critice

B1 — `sol17` citește mult în afara vectorului, din cauza unei împărțiri

`recapitulare-vectori.h:588`

CUM E ACUM: rulat pe un vector dimensionat exact (`int v[10]` , `m = 10`), sub AddressSanitizer:

```
ERROR: AddressSanitizer: stack-buffer-overflow
READ of size 4 ... in sol17(int, int*) recapitulare-vectori.h:588
```

DE CE: limita de sus e scrisă $2 * m - 1 / 7$. Spațiile te fac s-o citești ca $(2 * m - 1) / 7$, dar compilatorul nu vede spațiile — el aplică **întâi** împărțirea, deci calculează $2 * m - (1 / 7)$. Iar $1 / 7$ între două numere întregi e `0` . Limita devine $2 * m - 0$, adică **dublul** lungimii vectorului:

```
m = 10
m / 7 - 1    = 0    (start)
2 * m - 1/7  = 20    (stop)  <- ce calculeaza compilatorul
(2 * m - 1)/7 = 2    <- ce ai citit tu
```

Deci `i` merge de la `0` la `19`, iar în buclă citești și `v[i + 1]` — ajungi la `v[20]` pe un vector cu 10 elemente.

Capătul de jos are a doua problemă: `m / 7 - 1` dă `-1` pentru orice `m < 7`, iar `v[-1]` e tot în afara vectorului, doar că în partea cealaltă.

Împărțirea întreagă nu e „împărțire cu rotunjire”: `1/7` chiar **este** `0` în C++, nu `0.14`. De câte ori împărți numere întregi, întreabă-te ce iese când numărătorul e mai mic decât numitorul.

FIX: parantezele pe ce voiai să împărți, și o limită care nu poate ieși negativă. Dacă intervalul cerut e „ultima șeptime”, scrie-l explicit cu paranteze și verifică-l pe hârtie pentru `m = 5`, `m = 7`, `m = 20`.

B2 — `sol13` pornește de la `i = 1` și merge până la `i <= dim`

`recapitulare-vectori.h:456`

CUM E ACUM: `dim = 12`, deci indicii valizi sunt `0 – 11`. Bucla merge `1 – 12`: `v[0]` nu e verificat niciodată, iar `v[12]` e în afara vectorului. Pe un vector dimensionat exact (`int v[12]`), sub AddressSanitizer:

```
ERROR: AddressSanitizer: stack-buffer-overflow
READ of size 4 ... [32, 80) 'v' <== Memory access at offset 80
      overflows this variable
```

DE CE: aici nu e aceeași greșeală ca în `afisare` — pe aia ai reparat-o. Aici ai amestecat două lucruri care arată la fel: **indicele** din vector și **poziția** din cerință. Exercițiul vorbește despre „elementul de pe poziția `i`”, iar pozițiile se numără de la `1`. Indicii se numără de la `0`. Elementul de pe poziția `i` este `v[i - 1]`, nu `v[i]`.

Se vede că asta ai vrut, din faptul că folosești `i` și ca poziție: `cmmdc2(v[i], i)`. Dacă ai fi pornit bucla de la `0`, primul apel ar fi fost `cmmdc2(v[0], 0)` — iar `cmmdc2` cu `0` ca al doilea argument împarte la zero. Deci intuiția ta („pozițiile încep de la 1”) era bună; doar că ai mutat bucla în loc să muți indicele.

Și motivul pentru care n-a crăpat la tine: `v` e declarat `int v[100]` cu 12 valori, deci `v[12]` **există** și e `0`. Vectorul supradimensionat te ascunde de consecințe până în ziua în care îl dimensionezi exact.

FIX: `for (int i = 1; i <= dim; i++)` rămâne, dar înăuntru folosești `v[i - 1]`. Așa poziția e `i`, indicele e `i - 1`, și amândouă sunt corecte. (Vezi și M2: `sol15` și `ctNrPrimeNuCuPoz` pornesc tot

de la `i = 1` , dar acolo folosești `i` ca indice — deci acolo lipsește `v[0]` .)

B3 — `while (n > 10)` se oprește cu o cifră prea devreme

`recapitulare-vectori.h:172` (`UltimaPrimaCif`) · același bug la `:576` (`cifegale`)

Restanță: aceeași constatare ca B2 din runda 1 pe `recapitulare-cpp` .

CUM E ACUM:

apel	rezultat	corect
<code>UltimaPrimaCif(101)</code>	<code>false</code>	<code>true</code>
<code>UltimaPrimaCif(1001)</code>	<code>false</code>	<code>true</code>
<code>cifegale(100)</code>	<code>true</code>	<code>false</code>
<code>cifegale(10)</code>	<code>true</code>	<code>false</code>
<code>cifegale(1000)</code>	<code>true</code>	<code>false</code>

DE CE: bucla trebuie să ruleze **cât timp mai ai de tăiat cifre**, iar un număr mai are de tăiat exact cât timp are cel puțin două cifre — adică `n >= 10` . Cu `n > 10` te oprești pe `10` , care încă are două cifre.

La `UltimaPrimaCif(101)` : tai o dată, rămâi cu `10` , și `10 > 10` e fals. Deci compari ultima cifră `1` cu `10` , nu cu prima cifră `1` . La `cifegale(100)` : compari `0` cu ultima cifră a lui `10` , care e tot `0` , zici „egale” și ieși — fără să te uiți vreodată la `1` .

Toate cele cinci cazuri greșite au `0` sau `1` ca penultimă cifră. Nu e coincidență: bucla se oprește fix pe valoarea `10` , iar tu compari cu numărul `10` în loc de cifra `1` .

FIX: `while (n >= 10)` în ambele funcții.

B4 — `cifMinima(0)` întoarce `9`

`recapitulare-vectori.h:78`

Restanță: aceeași cauză ca B4 din runda 1 pe `recapitulare-cpp`.

CUM E ACUM: `cifMinima(0)` = 9. Cifra minimă a lui 0 e 0.

DE CE: `while (n > 0)` nu rulează **niciodată** pentru `n = 0`, deci funcția întoarce valoarea de pornire a lui `min`, adică santinela 9. Zero e singurul număr care n-are nicio cifră „mai mare ca 0” de tăiat, și de-aia e singurul care scapă din orice buclă scrisă cu `n > 0`.

Contează pentru vectori, nu e o chichiță: `cifMinimaVect` cheamă `cifMinima` pentru fiecare element. Un singur 0 în vector și rezultatul devine 9 în loc de 0 — adică exact pe dos.

FIX: `if (n == 0) { return 0; }` ca primă linie. Aceeași gardă și la `cifMax` dacă vrei simetrie (acolo iese 0 din întâmplare, pentru că santinela e 0).

B5 — `sol19` ignoră `m` și citește două poziții fixe

`recapitulare-vectori.h:648` · `recapitulare-vectori.cpp:77`

CUM E ACUM: `main()` citește `m` și apoi `m` valori. `sol19` cheamă `cmmmc(v[7], v[11])`, oricare ar fi `m`. Rulat cu intrarea 3 și 5 10 15, programul afișează 0 — `v[7]` și `v[11]` n-au fost niciodată citite. Compilatorul o spune direct:

```
recapitulare-vectori.h:647: warning: unused parameter 'm'
```

DE CE: un parametru nefolosit într-o funcție care primește și vectorul e aproape întotdeauna semnul că funcția lucrează pe altceva decât pe datele care i s-au dat. `int v[100]` din `main` are 100 de locuri, dar numai primele `m` au fost umplute de `cin`. Restul conțin ce era în memorie înainte — nu 0 garantat, ci orice.

La `m = 3` lanțul merge mai departe: `cmmmc(0, 0)` calculează `p / cmmdc(0, 0)`, iar `cmmdc(0, 0)` întoarce 0. Împărțirea la zero pe întregi n-are rezultat definit în C++ — pe calculatorul meu a ieșit 0, pe altul poate opri programul.

Și e singurul apel activ din `main()`, deci asta e tot ce face programul tău când îl rulezi acum.

FIX: ia pozițiile ca parametri sau verifică `m` înainte (`if (m < 12) return;`). Warning-ul „unused parameter” merită citit de fiecare dată — îți arată exact funcțiile care nu-și folosesc datele.

B6 — `cmmdc(n, 0)` se blochează la infinit

`recapitulare-vectori.h:434`

CUM E ACUM: `cmmdc(12, 0)` nu se termină. L-am omorât după 3 secunde.

DE CE: algoritmul cu scăderi repetate se oprește când `n == m`. Cu `m = 0`: `12 != 0`, deci `n = n - m = 12 - 0 = 12`. Nimic nu se schimbă, condiția rămâne adevărată, și bucla merge la nesfârșit. Scăderea micșorează numărul doar dacă ai ce scădea.

Nu e un caz teoretic: `cmmmc` cheamă `cmmdc`, iar `sol19` cheamă `cmmmc` pe valori **citite de la tastatură**. Un `0` tastat și programul îngheață fără niciun mesaj.

FIX: `if (n == 0) return m; if (m == 0) return n;` la intrarea în funcție. (`cmmdc2`, varianta cu rest, are aceeași problemă în alt loc: `n % m` cu `m = 0` împarte la zero.)

B7 — `cifreMijlocPatrate` are o ieșire fără `return`

`recapitulare-vectori.h:368-400`

CUM E ACUM: warning-ul de la compilare:

```
recapitulare-vectori.h:400: warning: non-void function does not return
a value in all control paths
```

DE CE: funcția se termină cu `if (x >= 100) { ... return ...; }` și nimic după. Tu știi că `x` e sigur `>= 100` acolo, pentru că ai tratat cazul `n < 100` mai sus și ai ieșit din funcție. Compilatorul **nu** face raționamentul ăsta: el vede o cale care ajunge la acolada de închidere fără `return`, și o semnalează.

Nu e o pedanterie. O funcție `bool` care se termină fără `return` nu întoarce `false` — nu întoarce **nimic definit**. Ce citește apelantul e ce s-a nimerit în registru, și poate fi `true` azi și `false` mâine.

FIX: scoate `if (x >= 100)` și lasă codul direct (condiția e deja garantată), sau pune `return false;` pe ultima linie.

B8 — `minElemVec` are santinela `9999`

`recapitulare-vectori.h:66`

CUM E ACUM: `minElemVec({50000, 20000, 30000}) = 9999`. Un număr care nici măcar nu e în vector.

DE CE: ai pornit căutarea de la o valoare aleasă „destul de mare”. Dacă toate elementele sunt mai mari decât ea, niciun `if` nu se execută și funcția întoarce santinela. Nu e o eroare de buclă, e o presupunere despre date care nu e scrisă nicăieri.

Varianta care nu poate greși: pornești de la **primul element**, pentru că el e sigur în vector. Minimul unui vector nevid e mereu unul dintre elemente — deci începe cu unul dintre ele.

FIX: `int min = v[0];` și bucla de la `i = 1`.

B9 — `descomp` nu afișează nimic pentru numere prime

`recapitulare-vectori.h:550` · și `:548`

CUM E ACUM:

```
descomp(24) -> 24=2^3+24=3^1+
descomp(27) -> 27=3^3+
descomp(13) ->                <- nimic, desi 13 = 13^1
```

DE CE: după bucla de împărțiri, ce rămâne în `n` e ultimul factor prim, și îl notezi corect cu `ct[n]++`. Dar bucla care afișează merge `for (int i = 2; i < x; i++)` — iar pentru un număr prim, factorul **este** `x`, și `x < x` e fals. Îl numeri și apoi nu te mai uiți la el. Aceeași formă ca B6 din runda trecută: capătul de sus al baleierii taie exact elementul care contează.

Al doilea lucru, mai grav și invizibil: `ct` are 10000 de locuri, iar `ct[n]++` scrie la indicele `n`. Pentru orice număr cu un factor prim `>= 10000` — de exemplu `descomp(20011)` — scrii în afara vectorului. Acolo nu e nimic al tău.

FIX: `i <= x` în bucla de afișare. Pentru `ct`, ține factorii într-un vector de perechi (factor, putere) de lungime mică, nu într-un vector indexat după valoarea factorului — orice număr poate avea un factor mai mare decât lungimea pe care ai ales-o.

B10 — `afisareNrRast` modifică vectorul primit

`recapitulare-vectori.h:203`

CUM E ACUM:

```
inainte : 607 6246 8 8932 7841 2024 1921 960 8 13
afiseaza: 6426 8 1291 8
dupa    : 607 6426 8 8932 7841 2024 1291 960 8 13
```

Două elemente s-au schimbat permanent în vectorul apelantului.

DE CE: `v[i] = rasturnat(v[i]);` scrie înapoi în vector, deși funcția se numește `afisare...`. Când treci un vector unei funcții în C++, nu se face nicio copie — funcția primește adresa și scrie direct în vectorul original. De-aia modificarea supraviețuiește după ce funcția se termină.

E periculos tocmai pentru că e tăcut: `sol5` afișează ceva corect, și abia exercițiul **următor** care folosește același vector primește date stricate. O funcție care afișează nu trebuie să lase urme.

FIX: `cout << rasturnat(v[i]) << " ";` fără atribuire. Și, ca promisiune verificată de compilator, `const int v[]` în antet — atunci scrierea nici nu compilează.

B11 — corpurile funcțiilor stau în `.h`

`recapitulare-vectori.h` (tot fișierul, 34 de funcții)

Restanță: aceeași constatare ca B10 din runda 1 pe `recapitulare-cpp`.

CUM E ACUM: merge, pentru că proiectul are un singur `.cpp`. Am adăugat un al doilea care include același header:

```
duplicate symbol 'afisare(int*, int)' in: recapitulare-vectori.o, al2lea.o
duplicate symbol 'sol1()' in: recapitulare-vectori.o, al2lea.o
ld: 2 duplicate symbols
```

DE CE: `#pragma once` te apără de includerea aceluiași header de două ori **în aceeași unitate de compilare**. Dar fiecare `.cpp` se compilează separat, iar `#include` înseamnă „copiază tot textul aici”. Două `.cpp` primesc fiecare câte o copie completă a celor 34 de corpuri, iar linkerul, care le pune cap la cap, găsește două definiții pentru același nume.

În `.h` doar **promisiunea** (`int cifMinima(int n);`), în `.cpp` **corpul**. Promisiunea se poate copia oricât; corpul trebuie să existe o singură dată.

La 648 de linii într-un singur fișier, asta a încetat să mai fie o chestie teoretică și a devenit motivul pentru care nu poți împărți tema pe fișiere.

FIX: `recapitulare-vectori.cpp` ia corpurile, headerul rămâne cu cele 34 de antete.

Importante

M1 — `forma2k` citește `k` de la tastatură, înăuntru. `recapitulare-vectori.h:505` . `sol15` cheamă `forma2k` pentru **fiecare** element par, deci programul cere câte un `k` nou la fiecare element. Rulat pe `4 8 3 16 5 32`, mi-a cerut trei valori. O funcție care răspunde la o întrebare („e `n` de forma `2^k` ?”) nu trebuie să și citească date; `k` e parametru: `bool forma2k(int n, int k)` . Citirea se face o dată, în `main()` .

M2 — `i = 1` ca indice, deci `v[0]` nu e verificat niciodată. `sol15:524` și `ctNrPrimeNuCuPoz:624` . Aici, spre deosebire de B2, `i` e folosit ca indice, nu ca poziție — deci lipsește pur și simplu primul element. Pe `4 8 3 16 5 32`, `sol15` nu se uită niciodată la `4` . (`sol16:562` pornește corect de la `1`, fiindcă acolo chiar ai nevoie de `v[i-1]` .)

M3 — `main()` = 50 de linii de apeluri comentate, cu unul singur activ. `recapitulare-vectori.cpp:8-77` . Exact obiceiul semnalat ca M4 în runda trecută, mutat în repo-ul nou. Problema practică: după ce reparați B3, n-ai cum să verificați dintr-o rulare că restul au rămas întregi. Scrie apelurile active, cu ce aștepți de la ele: `cout << "cifegale(100) " << cifegale(100) << " (astept 0)" << endl;`

M4 — `cifCons` acceptă cifre egale: `cifCons(555) → true` . :465 . Funcția cere ca fiecare cifră să fie `>=` vecina din stânga. Dacă cerința spunea „strict crescătoare” sau „consecutive”, e greșită — e aceeași constatare ca B8 din runda trecută (`cifCrescatoare` cu `<` în loc de `<=`). Spune-mi care era cerința exactă și o închidem într-un sens sau altul. Numele nu ajută: `cifCons` poate fi citit și „consecutive”, și „constante”.

M5 — `descomp` repetă numărul și lasă un `+` la final. :553 . `24=2^3+24=3^1+` în loc de `24=2^3*3^1` . Scrie `x=` o singură dată, înainte de buclă, și pune separatorul **între** factori (îl pui doar dacă nu ești la primul).

M6 — `nrPrim` (:27) și `ePrim` (:331) sunt aceeași funcție, scrisă de două ori. Diferă doar prin condiția de oprire (`d * d <= n` față de `d <= n / 2`) — prima e și mai rapidă. Ține una, cheam-o din

ambele locuri. Dacă găsești mâine un bug în logica de primalitate, cu două copii îl repari o dată și-l lași în picioare în cealaltă.

M7 — toți vectorii sunt `int v[100]` cu `dim` mult mai mic. Asta a ascuns B2 (`v[12]`) exista și era `0`) și ascunde orice depășire viitoare. Nu e o scăpare de stil: e motivul pentru care bug-urile de indice nu-ți dau nicio eroare la rulare.

Cleanups

- C1 — `using namespace std;` în header (`:3`). *Restanță din runda 1.*
 - C2 — de la `:400` încolo indentarea se pierde: corpurile funcțiilor nu mai sunt aliniate. Visual Studio reindentează tot fișierul cu `Ctrl+K` , `Ctrl+D` .
 - C3 — `ctNrPrimeNuCuPoz:624` are `i < +m` — un `+` rătăcit. Nu schimbă nimic, dar arată ca o greșeală de tastare rămasă în cod.
 - C4 — `nrPar` (`:494`) poate fi o linie: `return n % 2 == 0;` . Același lucru la `kDivPrp` , `UltimaPrimaCif` , `cifreMijlocPatrate` — toate au `if (conditie) return true; else return false;` , care **este** condiția.
 - C5 — nume: `UltimaPrimaCif` începe cu majusculă (restul sunt `camelCase`), iar `kDivPrp` și `cifCons` sunt prescurtate până nu se mai înțeleg. Scrie cuvinte întregi.
 - C6 — fișierul se termină fără linie nouă. *Restanță din runda 1.*
-

Before / After

#	Acum	Corect
B1	<code>i < 2 * m - 1 / 7</code>	<code>i < (2 * m - 1) / 7</code> , cu start ≥ 0
B2	<code>cmmdc2(v[i], i)</code> cu <code>i</code> de la 1	<code>cmmdc2(v[i - 1], i)</code>
B3	<code>while (n > 10)</code>	<code>while (n >= 10)</code>
B4	<code>int min = 9;</code> apoi <code>while</code> <code>(n > 0)</code>	<code>if (n == 0) return 0;</code> înainte
B5	<code>cmmmc(v[7], v[11])</code>	poziții ca parametri, sau gardă pe <code>m</code>
B6	<code>while (n != m)</code> fără gardă	<code>if (m == 0) return n;</code> înainte
B7	<code>if (x >= 100) { ... }</code> la final	fără <code>if</code> , sau <code>return</code> <code>false;</code>
B8	<code>int min = 9999;</code>	<code>int min = v[0];</code> , bucla de la 1
B9	<code>for (int i = 2; i <</code> <code>x; i++)</code>	<code>for (int i = 2; i <= x;</code> <code>i++)</code>
B10	<code>v[i] = rasturnat(v[i]);</code> <code>cout << v[i];</code>	<code>cout << rasturnat(v[i]);</code>
M1	<code>cin >> k;</code> în <code>forma2k</code>	<code>bool forma2k(int n, int</code> <code>k)</code>
M2	<code>for (int i = 1; i <</code> <code>m; i++)</code>	<code>for (int i = 0; i < m;</code> <code>i++)</code>

Întrebări

1. La B1, $2 * m - 1 / 7$ și $(2 * m - 1) / 7$ arată aproape la fel scrise, dar dau 20 și 2 pentru $m = 10$. Care e regula după care decide compilatorul, și de ce $1/7$ nu e 0.14 ?
2. La B2 ți-am arătat că bucla citește `v[12]` pe un vector cu 12 elemente — și totuși programul tău merge fără nicio eroare. Ce anume îl ține în picioare, și ce s-ar întâmpla dacă ai scrie `int v[12]` în loc de `int v[100]` ?
3. La B4 și la B3, ambele bug-uri sunt o buclă care se oprește cu un pas prea devreme (`n > 0` în loc de a trata 0 , `n > 10` în loc de `n >= 10`). Când scrii o buclă care taie cifre, care e întrebarea pe care trebuie să ți-o pui ca să nimeriști capătul din prima?



MyCodeSchool

Material didactic MyCodeSchool

mycodeschool.ro